The definition of a database could also be applied to a filesystem: databases are simply a structured set of data held on a computer. In a way, filesystems are also databases. However, when we talk about databases in terms of applications, we normally mean an RDBMS (Relational Database Management System) such as `mysql` or `mariadb`, although there are other types such as "NoSQL" databases like `mongodb` (which are unstructured document collections). Such systems do actually store data to the filesystem, ultimately, though not in formats intended to be readable by other software. They also offer a great deal of other functionality for ensuring consistency, replication (where multiple databases can be used to serve the same data), indexing (specialized data structures to greatly speed up searching), and aggregation (functionality for grouping, counting, sorting, and so on).

This section is not going to be a comprehensive guide to DB programming. That's a topic for an entire book, and indeed there are plenty of them that focus on just that. What we will be doing here is setting up a simple RDBMS (using Docker, of course!) and showcasing some of the tools that PHP offers for interacting with it. Let's start by bringing up a containerized instance of `mariadb` (a popular fork of the well-known `MySQL` RDBMS). We'll be using a `docker-compose.yaml` approach just as we did for `nginx` earlier, which is overkill because there's only one container being "orchestrated." You can work out the command line equivalent as an exercise, if you prefer!

One final word of warning: this section is going to use some very basic object-oriented features of PHP. If you're not comfortable with the concepts of objects or classes, you could always return to this part of the book after reading Chapter 6, "Object Fundamentals."

Launching the DB

First of all, we need a simple `docker-compose.yaml` file:

```
version: "3"
services:
  db:
    image: mariadb:lts
    container_name: mariadb
    ports:
      - "3306:3306"
```